



**UNIVERSIDAD
DE ANTIOQUIA**

1 8 0 3

Universidad de Antioquia
Facultad de Ingeniería
Dpt. De Electrónica y de Telecomunicaciones

Momento II Proyecto Final
Informatica II 2026-1

Realizado por:

Mateo Rodriguez Yepez
Emanuel Velez Arteaga

Docente:

Augusto Salazar

Mayo 05, 2026

1 Modo de Juego I

Head Football

Juego arcade que consiste en un 1 contra 1 que se caracteriza por dos personajes caricaturescos con una cabeza gigante "Cabezón"; todo este entorno ambientado en el icónico juego de Stardew Valley, la finalidad del juego es que se marque lo mas que se pueda en el tiempo desarrollado para poder ganar utilizando la cabeza y la bota.

1.1 Listado de requerimientos

Para el funcionamiento de este arcade el sistema debe de cumplir con los siguientes requerimientos:

- Movimiento del jugador: Lo jugadores deben moverse de izquierda a derecha y ejecutar saltos.
- Físicas del etorno: El entorno sistema para aplicar fuerza de gravedad sobre los personajes y la pelota.
- Sistema de colisiones: Sirve para calcular las colisiones entre la pelota y el jugador, además del limite del escenario.
- Sistema de puntuación: Para detectar cuando la pelota entra a la red y así contabilizar el punto o "Gol".
- Interfaz gráfica: Usando los recursos proporcionados por Qt como QGraphicsScene y QGraphicsView para que se renderice en 60 fps.

1.2 Objetos para el cumplimiento de los requerimientos

Se usa programación POO para el desarrollo de esta, con la cual se detallan las siguientes clases de momento:

- **Clase Player:** Encargada del desarrollo de lo que viene siendo la funcionalidad del personaje o "Cabezón".

Atributos

- Void Moverderecha(): Actualiza la posición X hacia la derecha.
- Void Moverizquierda (): Actualiza la posición Y hacia la izquierda.
- Void salto (): Aplicada con la fuerza de salto VelocidadY si Tierra es true.
- Void GravedadAplicada(): Resta un valor constante a velocidad.

Metódos

- Void Moverderecha(): Actualiza la posición X hacia la derecha.
- Void Moverizquierda (): Actualiza la posición Y hacia la izquierda.

- Void salto (): Aplicada con la fuerza de salto VelocidadY si Tierra es true.
- Void GravedadAplicada(): Resta un valor constante a velocidad
- **Clase Gamescene:** Controlador central de la ventana y bucle principal y los eventos de teclado.

Atributos

- * Player* player 1: Instacia del jugador 1.
- * Player* player 2: Instancia del jugador 2.
- * Ball* ball: Instacia del balón.
- * QTimer* gametimer: Temporizador que dicta los "Ticks" del juego

Metódos

- * Void conectar(): Conecta gametimer para actualizar de todas las físicas y lógicas.
- * Void TeclasPresionadas(): Verifica las teclas presionadas.
- * Void TecalsNoPresionadas (): Verifica si las teclas no están en uso.
- * Void Marcador (): Refresca el texto del marcador en la ventana.
- **Clase Ball:** Gestiona el comportamiento del balón.

Atributos

- float velocidadX: Velocidad de movimiento en el eje X.
- float velocidadY: Velocidad de movimiento en el eje Y.
- float Rebote: Coeficiente de restitución de cuanto rebota la pelota.
- float Gravedad: Aplicar

Metódos

- Void actualizacionDeposicion: Actualiza la posición del balón (X,Y) dadas las velocidades de la misma.
- Void VerificacionDecolision(): Verifica los rebotes si choca con un player.
- Void restaurarposicion (): Devuelve el balón al centro despues de una anotación.

1.3 Detalle de los Objetos

2 Modo de juego II

Paintball Game on the UdeA

Este juego consiste en un estilo similar a DOOM, en el que se disputa un juego de paintball entre el personaje principal (JP) y varios contrincantes, como: "los de arte", "ESMAD", "otakus" (posible modificación más adelante). Esta disputa de paintball estará ambientada en el entorno de la ciudadela universitaria de la UdeA. La finalidad es

culminar rondas sin recibir cierta cantidad de disparos por parte de los contrincantes.

2.1 Listado de requerimientos

Para el funcionamiento de este juego se deben cumplir los siguientes requerimientos:

- Raycasting: El raycasting es el principal requerimiento, porque es el que dará esa sensación de gráfico 3D. De hecho, así se llama: "raycasting" o "falso 3D". Este es el que generará la simulación y los gráficos en 3D
- Movimientos del jugador: Tanto el personaje principal como los contrincantes deben moverse en el espacio creado por el raycasting.
- Interacción entre los personajes: La interacción que ocurre al disparar, el impacto y la eliminación de cada personaje.
- Sistema de rondas: Para cada ronda habrá una cantidad de enemigos que derrotar, la cual se incrementa; y por cada enemigo los puntos deben aumentar.
- Gráficos, mapa y entorno: Aquí estará la parte en donde se debe simular el entorno de la UdeA, así como, para cada parte del mapa, qué textura tendrá, su profundidad y una proximidad a la ciudadela como tal.

2.2 Objetos y detalles para el cumplimiento de los requisitos

- **Clase Game:** Esta clase controlará el flujo del juego y coordina las demás clases.

Atributos

- * Player
- * Map
- * Raycaster

Metódos

- * init(): Encargado de iniciar todo
- * update(): Encargado de la lógica del juego

- **Clase Player:** Esta es la clase encargada del personaje principal y los contrincantes

Atributos

- * Float x,y → La posición.
- * Float angle → La dirección
- * Float speed → la velocidad

Metódos

- * moveforward() → Movimiento
- * movebackward() → Movimiento
- * rotateleft() → Rotación del jugador

- * rotateright() → Rotación del jugador
- * update()
- **Clase Map:** Esta clase es la encargada de darle color y las características al entorno requerido.

Atributos

- * matriz → (Mediante puntero o stl como vector)
- * int width,height

Metódos

- * iswall() → colisión de paredes
- * chackcollision()

- **Clase Raycaster:** Esta es la clase encargada de la implementación del 3D

Atributos

- * int numrays
- * float fov → campo de visión
- * float maxdepth

Metódos

- * castrays()
- * castsingleray()
- * getdistance()

Esta es una aproximación a lo que respecta al desarrollo del proyecto final. Está sujeta a modificaciones tanto en la estructura como en la lógica de cada punto del proyecto. Esta entrega es una previsualización de hacia dónde está yendo el proyecto.